

BudgetBridge - Milestone 02 Database Design Package

CP476B Group 6 - Personal Expense Tracker

Due Date: June 26, 2026

1. Database Design Scope

This package contains the database design required for Milestone 02: a relational diagram and SQL CREATE TABLE statements with primary keys, foreign keys, constraints, and sensible MySQL data types. The design supports the core BudgetBridge workflow: account access, category management, transaction CRUD, filtering, searching, dashboard totals, and category/date-range reports.

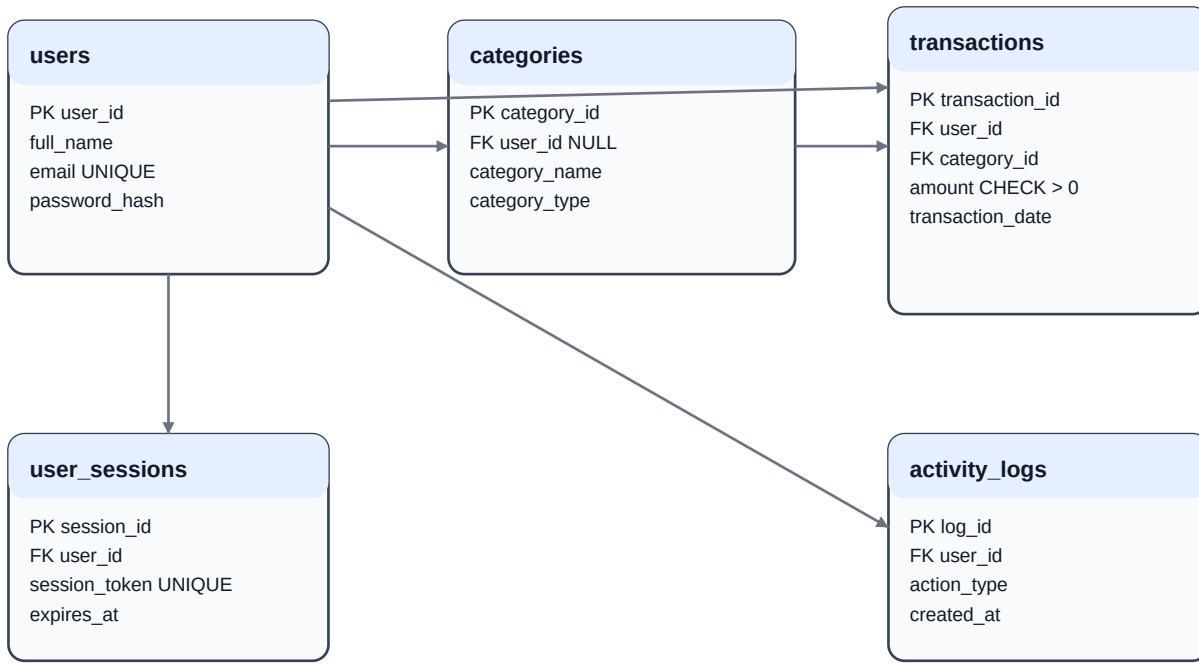
Table	Purpose	Main Keys / Constraints
users	Stores registered user accounts.	PK user_id; UNIQUE email; password_hash required
categories	Stores default and user-created income/expense categories.	PK category_id; FK user_id; UNIQUE user/category pair
transactions	Stores income and expense records.	PK transaction_id; FK user_id; FK category_id; amount CHECK > 0
user_sessions	Supports login session management for the back-end skeleton.	PK session_id; FK user_id; UNIQUE session_token
activity_logs	Records important user actions for tracking and debugging.	PK log_id; FK user_id

2. High-Level Relationships

- One user can have many transactions; each transaction belongs to one user.
- One user can have many categories; each category may be default or user-created.
- One category can be used by many transactions.
- One user can have many session records.
- One user can have many activity log records.

3. Relational Diagram

Primary keys are marked with PK and foreign keys are marked with FK. The diagram below shows the main one-to-many relationships used by the application.



Relationship pattern: users own transactions, categories, sessions, and logs. Categories classify transactions.

4. Normalization and Constraint Notes

Design Choice	Reason
Separate users table	Keeps account data independent from transactions and supports user-owned records.
Separate categories table	Avoids repeating category names directly inside every transaction record.
transactions references users and categories	Supports ownership checks, filtering, dashboard totals, and category reports.
CHECK constraints and NOT NULL fields	Prevents invalid records such as zero/negative transaction amounts or missing required values.
UNIQUE email and session_token	Prevents duplicate accounts and duplicate active session tokens.
Indexes on transaction filters	Improves common queries by user, date, category, and type.

5. SQL CREATE TABLE Statements

The following SQL is also provided separately in database/schema.sql in the GitHub repository.

```
-- BudgetBridge Personal Expense Tracker
-- CP476B Milestone 02 Database Schema
-- Target DBMS: MySQL 8.x

CREATE DATABASE IF NOT EXISTS budgetbridge_db;
USE budgetbridge_db;

CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL,
    category_name VARCHAR(80) NOT NULL,
    category_type ENUM('income', 'expense') NOT NULL,
    is_default BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_categories_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE CASCADE,
    CONSTRAINT uq_category_per_user
        UNIQUE (user_id, category_name)
);

CREATE TABLE transactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    category_id INT NOT NULL,
    transaction_type ENUM('income', 'expense') NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    transaction_date DATE NOT NULL,
    description VARCHAR(255),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT chk_transaction_amount_positive CHECK (amount > 0),
    CONSTRAINT fk_transactions_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_transactions_category
        FOREIGN KEY (category_id) REFERENCES categories(category_id)
        ON DELETE RESTRICT
);

CREATE TABLE user_sessions (
    session_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    session_token VARCHAR(255) NOT NULL UNIQUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP NOT NULL,
    CONSTRAINT fk_sessions_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE CASCADE
);

CREATE TABLE activity_logs (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    action_type VARCHAR(50) NOT NULL,
    action_description VARCHAR(255) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_logs_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE CASCADE
);

CREATE INDEX idx_transactions_user_date ON transactions(user_id, transaction_date);
CREATE INDEX idx_transactions_user_category ON transactions(user_id, category_id);
```

```
CREATE INDEX idx_transactions_type ON transactions(transaction_type);
CREATE INDEX idx_categories_type ON categories(category_type);
CREATE INDEX idx_activity_logs_user_created ON activity_logs(user_id, created_at);

-- Default categories can be inserted for each new user by server-side application logic.
-- Example default categories: Income, Food, Rent, Transportation, School, Entertainment, Other.
```